ABINASH DEVENDHIRAN

Senior Software Developer | AI-steered Engineering

abiabinash1323@gmail.com | +91 90804 76982 | Chennai, India

github.com/Abinash0416

https://abinash0416.github.io

linkedin.com/in/abinash-devendhiran

SUMMARY

Full Stack Developer and Technical Lead with 5+ years of experience building and shipping products. Led delivery across multi-codebase platforms, managed a team of developers, and built data pipelines, marketplaces and mobile apps, owning system design from architecture to deployment. Work AI-first: AI-driven architecture, design improvements and PR reviews, and complex applications developed with AI assistance. Built MCP servers with FastMCP and use Claude Code agents and skills in day-to-day delivery.

SKILLS

Agentic Development: Claude Code (Agents, Artifacts, Skills, Hooks, Workflows), MCP Servers (FastMCP, Penpot, Jira, Atlassian, Postman), Multi-Agent Orchestration, LLM APIs (OpenAI, Anthropic)

Backend: Ruby on Rails (Ruby, Cable, Sidekiq, Devise, Pundit, RSpec), Node.js (JavaScript, TypeScript, Express), Django (Python, DRF, SQLAlchemy, Celery, pytest), FastAPI (Alembic), Go, Rust, C#, WebSocket, REST API Design, JWT Auth and RBAC, Webhooks, Background Jobs

Frontend and Mobile: React (Jest/Vitest, Playwright), Next.js, Vue, Nuxt, Tailwind, Redux, Vite, Flutter (Riverpod, Bloc), Kotlin (Android, ADB on physical and virtual devices), Swift (iOS, TestFlight)

Database & Infra: PostgreSQL (SQL), ClickHouse, Apache Druid, Elasticsearch, Redis, SQLite, MongoDB, Docker, MinIO, Railway, Firebase, AWS (S3), OpenVPN

Integrations: Razorpay, Stripe, WhatsApp API, FCM, Google Maps, SendGrid, Slack API, Dropbox API, Google Calendar API, Google OAuth, Keycloak (SSO), reCAPTCHA v3, Kafka, Swagger/OpenAPI

Practices: System Design, Database Design, Web Scraping, Data Pipelines, Role-Based Access, Team Leadership, Code Review (AI-assisted, custom Claude Code skill)

WORK EXPERIENCE

Solution Consultant - Francium Tech

May 2021 - Present

Teldat VPN · Technical Lead

Oct 2026 - Present

Multi-tenant VPN security platform in four codebases (BeSafe Pro, beSafeAgent, Windows agent, besafe-argus): admins set per-user profiles that agents enforce over OpenVPN, with Argus malware scanning and telemetry.

- **Requirements and delivery:** gathered requirements from the client, presented them as work items, assigned them to developers and maintained them through delivery. Reviewed code both manually and with AI assistance, and built a custom Claude Code skill for pull request review. Wrote code hands-on, including scheduled jobs.
- **Team management and code review:** led junior developers, assigning work, running daily stand-ups and tracking progress through to delivery. Coached them in coding discipline and brought a structure and pipeline to the whole process, from requirements and pull request review through to release.
- **Argus and network telemetry:** Argus updates roll out with install approval and auto-apply after 2 days, and protection works in split-tunnel and no-VPN modes, with per-user licence (`BS_ARGUS`), admin-forced or optional mode, expiry jobs and reconnect after sleep and power-offs. NetFlow and syslog (CEF, Suricata EVE, RFC 5424) are per-user controls with multiple collectors, monitoring all traffic in any VPN state via an Android capture VPN.

Stack: Ruby on Rails, Sidekiq, React, Go, ClickHouse, Rust, Tauri, Kotlin (JNI), C# (.NET 8), SQLCipher, OpenVPN, Docker

Golden Set Analytics · Data Engineer

Jan 2026 - Sep 2026

Tennis analytics platform for rankings, stats and scouting. Django REST Framework on PostgreSQL: Celery Beat scrapers land raw rows idempotently in intake, and promotion resolves them into one bedrock dataset.

- **Scrapers:** built and maintained scrapers for CoreTennis, ITA, ITF, WTN, ATP/WTA, Tennis New Zealand and TennisExplorer with requests, BeautifulSoup and Playwright. Each command has run modes: scrape-only into intake (`--no-propagate`) or push on to bedrock, `--dry-run`, and token-bucket rate limiting with worker pools. Runs can be targeted by tournament ids, tier, tour, age band or team roster, and full-archive sweeps resume from a checkpoint file.
- **Backfill workflow design:** snapshot, delete, re-scrape, re-promote workflow with per-batch transactions, guarded by a required btree index on `intake_match.bedrock_tennis_match_id` (BRIN cost 2,119 ms vs 0.55 ms).

- **Player data consolidation and country normalisation:** anchored players to immutable source-of-truth details and merged records across sources, using per-source trust scores (data purity) to resolve conflicts. Built a country registry of 250 countries (ISO3, IOC, FIFA) so each source's code format resolves correctly.
- **Protected-site scraping:** fixed ATP date-of-birth enrichment, which had never succeeded behind Cloudflare, by driving headless Chromium with one browser context reused across retries to keep the clearance, detecting the interstitial and re-navigating, and pacing profile fetches with exponential cooldown. Top 100 went from 0 to 100 enriched in 14 minutes instead of 2 hours, and a 72-snapshot historical sweep resolved 98.5%.
- **State machine repair:** repaired about 178.7k pre-linked intake players stuck at `new` with a read-only sweep that re-ran the production matcher in dry-run under a read-only transaction, bucketing results as same, different, would-create or review. The write steps (apply, rescrape, relink) take human-reviewed CSVs, and 104 flagged rows were held back.

Stack: Python, Django, DRF, PostgreSQL (pg_trgm), Celery Beat, Redis, Playwright, BeautifulSoup, Docker

Atrium · Technical Lead

Jul 2025 - Dec 2025

Parental-control and screen-time app for Android and iOS where parents set app, category and website limits for children. Rails 8 API on PostgreSQL, Flutter app, Kotlin on Android, Swift Screen Time extensions on iOS.

- **Architecture and team leadership:** initiated the project from scratch, then improved and finalised the base design across backend, mobile and web, and defined a Flutter platform-dispatch pattern where UI code never branches on OS (one dispatcher per feature routes to per-platform implementations). Led a team of developers across Rails, Flutter, Kotlin, Swift and React, reviewing pull requests and driving the beta release.
- **Authentication and authorization:** designed JWT authentication with email, password reset, and Google and Apple sign-in (OIDC ID tokens verified against JWKS keys with issuer and audience checks), registering a device on every login. Role-based access (CanCanCan): parents manage rules, devices and usage of their children, and self-managed users manage their own. Rules need a shared family, and children cannot set their own.
- **Android:** `UsageStatsManager` tracking synced by WorkManager (keyed by `ANDROID_ID`), limits enforced locally every 5 seconds in a foreground task, and an Accessibility Service blocking apps and browser domains.
- **Rule engine and delivery:** a single `BlockedPackagesCalculator` combines limit rules (daily, once, weekday and date schedules), usage, categories and approved extra-time grants into blocked packages and domains, with approved extra time expiring through a scheduled job.
- **iOS challenges:** iOS cannot list apps, read usage or control other apps like Android, each solved differently: apps are picked via `FamilyActivityPicker` and stored as opaque tokens, usage comes from Device Activity threshold events, and control from Family Controls and Managed Settings shields, shared via an App Group.

Stack: Ruby on Rails 8, PostgreSQL, Flutter, Kotlin, Swift, React, FCM, ActionCable, Docker

Pan Agro · Lead Developer

Feb 2024 - Jun 2025

Spain-based agri marketplace where farmers list produce, retailers place requests and transporters bid tender-style for delivery. Expo apps run on a Rails and Beego backend with PostgreSQL, Redis-backed workers, Kafka and MinIO, with market data in Apache Druid and ARIMA price forecasts from FastAPI behind a time-series model selector.

- **Architecture and DB design:** set the initial architecture and designed the database structure across 100+ models covering offers, requests, negotiations, orders, shipments, transport, crops, insurance and finance, with Swagger APIs.
- **Marketplace and transport tendering:** production offers, buyer requests, negotiations, orders and shipments for producers, buyers, suppliers and transporters, with in-app chat and notifications pushed over WebSocket and email. Transporters bid on deliveries, and an accepted offer waits for the blockchain contract creation before it is confirmed and generates the transport contract PDF from the PIDA template.
- **Market data and forecasting:** built the LSEG and MAPA market-history population APIs and scheduled MAPA jobs into Apache Druid and PostgreSQL, with units per curve. Wired ARIMA predictions through FastAPI and the model selector's per-source predict endpoint, resolving PIDA-DRUID curve ids from product ids.
- **VDR, weather and crops:** role-checked multi-file upload to MinIO (certificates producer-only, insurance files in their own bucket), plus weather data and crop growth stages per region for the farmers' dashboards.
- **Secure engine:** shipped the core logic (authentication and role permissions, insurance premiums, price forecasting, blockchain contract calls, chat) as a compiled Go executable, so the client receives no source and our IP and copyright stay protected. The API runs it as a child process over a JSON stdin/stdout protocol, with health checks.
- **Team and delivery:** reviewed pull requests and regularly handled people management for the team, improved and approved the UI, and created and maintained the tickets.

Stack: Go, Ruby on Rails, React, React Native, PostgreSQL, Apache Druid, FastAPI, MinIO, Docker

FREELANCE PROJECTS

Magizh Foods · Full Stack Developer

Feb 2026 - Present

Platform for a cloud kitchen with home delivery. Customers order one-time or subscribe to variety of plans, and the kitchen runs everything from an admin portal.

- **Solo build and four user types:** Rails 8 API (Solid Queue, Solid Cache, Solid Cable), a React 19 and TypeScript web app and a Flutter mobile app, both with desktop, tablet and phone layouts, on Railway. UI and UX were designed first as Claude artifacts prototypes. Admin, customer, kitchen worker and delivery partner each have their own flow, enforced with CanCanCan, JWT sessions, Rack::Attack throttling and AES-encrypted role names.
- **Orders, payments and wallet design:** one-time and subscription orders with a different address per meal. Razorpay with signature-verified webhooks and refunds, a per-minute job that expires unpaid orders, and WhatsApp approve and reject buttons for admins. A cancelled delivery becomes a wallet credit with a validation hash, spent oldest first under row locks, with the balance computed in SQL and never stored.
- **Flexible plans and ordering:** admins create recurring plans (any mix of meal days, shifts and durations, priced per duration and shift) or one-time event plans with specific dates and per-date prices. Customers choose their own days, shifts and duration, and each shift can have its own address. Every plan edit saves a read-only snapshot, so existing orders keep their prices and meals.
- **Coupon and referral systems:** percentage (capped) or flat coupons with per-user, global user and budget limits, and rules for date range, nth subscription, minimum cart, plan and duration. Validation runs in a fixed order with a preview mode, and the coupon row is locked at checkout to stop overspending. Referrals reward on signup or first paid subscription via share link or mobile deep link, with self-referral blocked.
- **Delivery management:** a staff board (ongoing, upcoming, past and calendar views) assigns meals to partners in batches inside the shift window. Partners mark delivered or revoke within 60 minutes of assignment, and updates reach every screen live over Solid Cable and FCM push.
- **Leave announcements and adjustment:** an announced kitchen leave cancels the pending recurring meals in its range and appends replacement meals to each subscription in one transaction, with an impact preview. Deleting the leave restores them, and it is blocked if one-time meals or another leave fall in the range.
- **Meal skipping and reminders:** a skipped meal is added to the end of the subscription or refunded to the wallet. Recurring jobs queue a WhatsApp reminder with a Skip button, stamped per meal so re-runs never double-send.

Stack: Rails 8, Solid Queue, PostgreSQL, React, TypeScript, Flutter, Razorpay, WhatsApp API, FCM, Google Maps, Railway

Fampic Registry · Full Stack Developer

Sep 2026 - Oct 2026

Serverless work and payment registry for a photo studio. Every phone and computer holds a full copy of the data and syncs.

- **Device discovery:** UDP broadcast and multicast on the same WiFi. Over the internet, a beacon derived from the group secret: 4 Ed25519-signed pkarr slots on the n0 relay, written every 5 minutes and read every 30 seconds. Devices dial what they find over iroh, with hole punching and relay fallback.
- **Sync protocol design and recovering missed data:** every event is identified by `(deviceId, seq)`, a per-device counter that only increases. Devices swap a hello with their version vector (highest seq seen per device), and each sends only the events the other is missing, in chunks of 200. Events apply in seq order per device in one transaction with the vector update. Seen events are skipped, and a seq gap triggers a fresh hello so the peer resends from the missing point, with nothing lost or duplicated.
- **Live propagation:** a new event is pushed to every connected peer and forwarded onward except to the sender, only if it is new to that device, so propagation stops by itself and reaches devices that never meet the author. Links ping every 5 seconds, drop after 15 seconds of silence, and known peers are redialed every 15 seconds.
- **Conflict resolution:** Lamport clocks with a deviceId tie-break and tombstones for deletes, so every device converges whatever order events arrive in, with no wall-clock time. A pending payment's remaining amount is computed from the payments pointing at it, so offline part payments cannot overwrite each other.
- **Security:** events sealed with a shared group key (AES-GCM), logins locked by Argon2id-derived keys with no password hash stored, and an Ed25519 signature on every event. Role rules apply when tables are rebuilt, so events without the role stay in the log but are ignored.

Stack: Flutter, Dart, C#, WPF, Swift, SwiftUI, Rust, iroh, SQLite

EDUCATION

B.Sc Computer Science, J.H.A Agarsan College, Madras University, Chennai, TN 2017 - 2020

LANGUAGES

Tamil (Native), English (Professional)